Processing Time versus Circuit Size in a 32-Bit FPGA GCD Unit

Ahmed Saad Sweffi
Digital Design & Verification Engineer
RTL · FPGA · VLSI

***Abstract*—The task was to design an FPGA circuit that computes the greatest common divisor (GCD) of two 32-bit integers while keeping both the processing time and the circuit size small. Since these two goals work against each other, I built two Verilog cores with the same interface and compared them: one based on Stein's binary algorithm and tuned for speed, and one based on Euclid's subtraction algorithm in a compact FSMD form and tuned for size. Both give $\gcd(14, 161) = 7$, in 7 and 18 clock cycles respectively, and both pass the same 85,565 test vectors. On a Zybo Z7-20 board, one call takes 1,180 ns with the Stein core and 2,173 ns with the FSMD core. The Stein core never needs more than 64 cycles, while the FSMD core can need about 4.29×10^9; in return, the FSMD core uses 58% fewer LUTs and 48% fewer slices.**

***Index Terms*—GCD, Stein's algorithm, FSMD, FPGA, Verilog, Zynq-7000, AXI4-Lite, area–time trade-off**

I. Introduction

The task I worked on was the following:

> *"Write the VHDL code of a circuit (FPGA) that computes the greatest common divisor (GCD) between two 32-bit integers. If possible, please report the simulation results when your circuit computes the GCD of 14 and 161. Think at optimizing the processing time and the circuit size as much as you can."*

The task asks for VHDL, but I wrote the design in Verilog, the hardware description language (HDL) I prefer. Both languages describe the same kind of synthesizable hardware and go through the same Vivado flow, so this choice does not change any of the results.

The last sentence of the task is the interesting one, because processing time and circuit size usually pull against each other. A faster circuit does more work in each clock cycle and therefore needs more logic, while a smaller one has to spread the work over more cycles. Instead of settling on a single compromise, I designed two architectures, one at each end of this trade-off, and compared them under the same conditions: `gcd32`, based on Stein's binary algorithm, aims at processing time, and `fsmd_gcd32`, based on Euclid's subtraction algorithm and built as a finite-state machine with datapath (FSMD), aims at circuit size. Both cores have the same ports and handshake (`start`, `busy`, `done`, `gcd_out`), so either one can be dropped into the same system.

TABLE I
One step of Stein's algorithm, executed by `gcd32` in each clock cycle

Condition	Register update
$a = b$	done, $gcd \leftarrow a \ll k$
a and b even	$a \leftarrow a \gg 1$, $b \leftarrow b \gg 1$, $k \leftarrow k + 1$
a even, b odd	$a \leftarrow a \gg 1$
a odd, b even	$b \leftarrow b \gg 1$
both odd, $a > b$	$a \leftarrow (a - b) \gg 1$
both odd, $a < b$	$b \leftarrow (b - a) \gg 1$

II. Algorithms and Architectures

A. Choosing the Algorithms

The textbook way to compute a GCD is Euclid's algorithm, $\gcd(a, b) = \gcd(b, a \bmod b)$. It needs only a few iterations, but every iteration needs a 32-bit division, and a divider is large and slow on an FPGA. If the modulo is replaced by repeated subtraction, the datapath becomes tiny, just one subtractor, but the number of iterations now depends on the quotients and can reach about 2^{32}, for example for $\gcd(2^{32} - 1, 1)$. Stein's binary algorithm [1], [2] avoids division altogether: it only halves numbers, which costs nothing but wiring in hardware, subtracts once per step, and always finishes within $2n$ steps for n-bit operands. Systolic arrays [3] go faster still by pipelining, but at a much larger area. This settled the choice for my two designs: the binary algorithm for speed and Euclid by subtraction for size.

B. `gcd32`: Stein's Binary Algorithm (Processing Time)

`gcd32` (Listing 1) performs one step of Stein's algorithm in every clock cycle, following the rules in Table I. A small counter k remembers how many factors of two the operands had in common, and a final shift puts them back into the result. To keep the critical path short, the core computes $a - b$ and $b - a$ at the same time and uses the borrow of $a - b$ to pick the right one, so there is no compare-then-subtract chain. If one operand is zero, the answer is simply the other one, and the core returns it at once. Because every step removes at least one bit from a or b, two 32-bit numbers never need more than 62 steps. With the load and the final comparison, that makes at most **64 cycles**, which is exactly what $\gcd(2^{32} - 1, 2^{32} - 2)$ takes.

Listing 1. `gcd32.v`: Stein's binary GCD, one step per clock cycle

```
module gcd32 (
    input  wire        clk,
    input  wire        rst_n,      // active-low synchronous reset
    input  wire        start,
    input  wire [31:0] a_in,
    input  wire [31:0] b_in,
    output reg         busy,
    output reg         done,
    output reg  [31:0] gcd_out
);

    reg [31:0] a, b;
    reg [4:0]  k;                   // common power of two (max 31)

    // Two subtractors in parallel: the sign of (a-b) selects which result is
    // used, avoiding a serial compare->subtract path (shorter critical path).
    wire [32:0] a_minus_b = {1'b0, a} - {1'b0, b};
    wire [31:0] b_minus_a = b - a;
    wire        a_lt_b    = a_minus_b[32];        // borrow => a < b
    wire        eq        = (a == b);

    always @(posedge clk) begin
        if (!rst_n) begin
            a       <= 32'd0;
            b       <= 32'd0;
            k       <= 5'd0;
            busy    <= 1'b0;
            done    <= 1'b0;
            gcd_out <= 32'd0;
        end else begin
            done <= 1'b0;

            if (start && !busy) begin
                // Zero operands are resolved immediately (no iterations).
                if (a_in == 32'd0 || b_in == 32'd0) begin
                    gcd_out <= a_in | b_in;
                    done    <= 1'b1;
                end else begin
                    a    <= a_in;
                    b    <= b_in;
                    k    <= 5'd0;
                    busy <= 1'b1;
                end
            end else if (busy) begin
                if (eq) begin
                    gcd_out <= a << k;      // restore the common 2^k factor
                    busy    <= 1'b0;
                    done    <= 1'b1;
                end else begin
                    case ({a[0], b[0]})
                        2'b00: begin        // both even
                            a <= a >> 1;
                            b <= b >> 1;
                            k <= k + 5'd1;
                        end
                        2'b01: a <= a >> 1; // a even
                        2'b10: b <= b >> 1; // b even
                        2'b11: begin        // both odd
                            if (a_lt_b) b <= b_minus_a     >> 1;
                            else        a <= a_minus_b[31:0] >> 1;
                        end
                    endcase
                end
            end
        end
    end

endmodule
```

C. `fsmd_gcd32`: *Euclid's Subtraction Algorithm as an FSMD (Circuit Size)*

`fsmd_gcd32` (Listing 2) is written in the classic FSMD style [4]: a small controller tells a datapath what to do in each cycle. The datapath holds only the registers a, b and `gcd_out`, a *single* 33-bit subtractor, whose borrow doubles as the $a < b$ comparison, and a check for $b = 0$. The controller has just two states, IDLE and CALC, and in every cycle it picks one operation: LOAD the operands, SWAP them when $a < b$, SUBtract b from a, or FINish once b reaches zero. The price of this simplicity is the latency, which equals $2 + \sum_i (q_i + 1)$ cycles, where q_i are the quotients of Euclid's algorithm. That is only 18 cycles for $\gcd(14, 161)$, but $2^{32} + 4 \approx 4.29 \times 10^9$ cycles, about 43 s at 100 MHz, for $\gcd(2^{32} - 2,\ 2^{32} - 1)$. To save space, the listings leave out the long comment blocks of the source files.

Listing 2. `fsmd_gcd32.v`: Euclid by repeated subtraction, two-state FSMD

```
module fsmd_gcd32 (
    input  wire        clk,
    input  wire        rst_n,      // active-low synchronous reset
    input  wire        start,
    input  wire [31:0] a_in,
    input  wire [31:0] b_in,
    output wire        busy,
    output reg         done,
    output reg  [31:0] gcd_out
);

    // ----------------------------------------------------------------------
    // Datapath: operands and status signals
    // ----------------------------------------------------------------------
    reg  [31:0] a, b;

    wire [32:0] a_minus_b = {1'b0, a} - {1'b0, b};
    wire        a_lt_b    = a_minus_b[32];        // borrow => a < b
    wire        b_zero    = (b == 32'd0);         // loop finished

    // ----------------------------------------------------------------------
    // Controller
    // ----------------------------------------------------------------------
    localparam S_IDLE = 1'b0,
               S_CALC = 1'b1;

    reg state, state_next;
    reg load, swap, sub, fin;                     // control signals (one-hot)

    always @* begin
        state_next = state;
        {load, swap, sub, fin} = 4'b0000;
        case (state)
            S_IDLE:
                if (start) begin
                    load       = 1'b1;
                    state_next = S_CALC;
                end
            S_CALC:
                if (b_zero) begin
                    fin        = 1'b1;
                    state_next = S_IDLE;
                end else if (a_lt_b)
                    swap = 1'b1;
                else
                    sub  = 1'b1;
        endcase
    end

    always @(posedge clk) begin
        if (!rst_n) begin
            state <= S_IDLE;
            done  <= 1'b0;
        end else begin
            state <= state_next;
            done  <= fin;
        end
    end

    assign busy = (state == S_CALC);              // the state flip-flop itself

    // ----------------------------------------------------------------------
    // Datapath registers
    // ----------------------------------------------------------------------
    always @(posedge clk) begin
        if (!rst_n) begin
            a       <= 32'd0;
            b       <= 32'd0;
            gcd_out <= 32'd0;
        end else begin
            if (load) begin
                a <= a_in;
                b <= b_in;
            end else if (swap) begin
                a <= b;
                b <= a;
            end else if (sub) begin
                a <= a_minus_b[31:0];
            end

            if (fin)
                gcd_out <= a;
        end
    end

endmodule
```

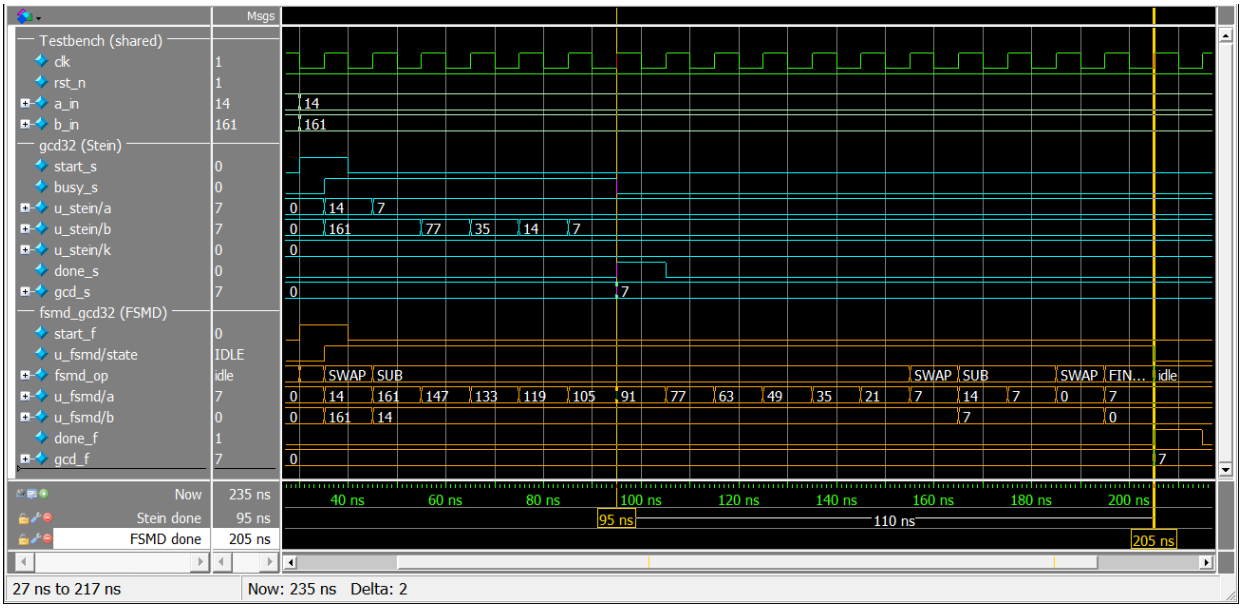


Fig. 1. Simulation waveform of the required test, $\gcd(14, 161)$: `gcd32` (Stein, cyan) and `fsmd_gcd32` (FSMD, orange) start on the same clock edge at 35 ns, and the `fsmd_op` row shows the FSMD operation in each cycle. The cursors mark the `done` pulses: 95 ns for Stein (7 cycles) and 205 ns for the FSMD (18 cycles).

III. Simulation and Testbench

To compare the two cores fairly, I tested them together in one self-checking testbench. Both cores share the same 100 MHz clock, reset and operand inputs, so every test vector reaches both of them on the same clock edge. For each vector, the testbench compares both results with a software reference and checks both latencies against a cycle-by-cycle model of each core. Besides the required case $\gcd(14, 161)$, the vectors cover corner cases (zero and equal operands, powers of two, the largest 32-bit values), all 65,536 pairs of 8-bit operands and 20,000 random pairs. **All 85,565 vectors pass on both cores.** A few FSMD cases would take more than 2^{24} cycles to simulate, so for those the testbench relies on the cycle model, which the board later confirmed on runs of up to 4.29×10^9 cycles.

Fig. 1 shows the required test. Both cores see `start` at 35 ns. The Stein core halves a from 14 to 7, brings b down from 161 to 77, 35 and 14 in three both-odd steps, and halves it to 7. Now $a = b$, so at 95 ns it raises `done` with `gcd_out` $=$ 7: **7 cycles**, or 70 ns. The FSMD core swaps the operands, subtracts 14 eleven times to get from 161 down to 7, swaps, subtracts twice, swaps once more and finishes at 205 ns, also with `gcd_out` $= 7$, after **18 cycles** (180 ns). For this example, the Stein core is 2.57 times faster.

IV. FPGA Deployment and Block Design

For the hardware tests I used a Digilent Zybo Z7-20 board [5]. Its Xilinx Zynq-7020 (XC7Z020-1CLG400) combines a dual-core ARM Cortex-A9 processor, the processing system (PS), with FPGA fabric, the programmable logic (PL) [6]. I built the system with Vivado and Vitis 2021.1.

AXI connection. To let the processor talk to a GCD core, I wrapped each core in a small AXI4-Lite slave [7] that turns it into a set of 32-bit registers: A and B for the operands, CTRL to start, STATUS to see whether the core is busy or done, RESULT for the answer, CYCLES for the measured latency, and ID to check that the right bitstream is loaded. The FSMD wrapper has one extra register for the upper half of the cycle count, because its latency can exceed 2^{32} cycles. I packaged the wrapper as an IP and connected it in a Vivado block design [8] generated by a Tcl script (Fig. 2). The processor reaches the core through its `M_AXI_GP0` port and an AXI Interconnect at address 0x43C0_0000, and a Processor System Reset block ties everything to the 100 MHz fabric clock. From the software side, one GCD is just a few register accesses: write A and B, write START, wait until STATUS reports DONE, and read RESULT.

Board validation. A bare-metal program on the ARM core ran 18 hand-picked and 10,000 random operand pairs and compared every hardware result with a software reference. Both designs passed all 10,018 cases, and the cycle counts read back from the board match the simulation exactly (7 and 18 cycles for $\gcd(14, 161)$). Timed over 10,000 calls, one call takes **1,180 ns** with the Stein core and **2,173 ns** with the FSMD core. Roughly 0.8 µs of every call goes into the AXI register accesses themselves, and that part costs the same for both cores.

V. Results and Comparison

Table II puts the two designs side by side. Processing time is counted in cycles of the 100 MHz clock, where simulation and board agree cycle for cycle. Circuit size comes from synthesizing, placing and routing each core on its own in

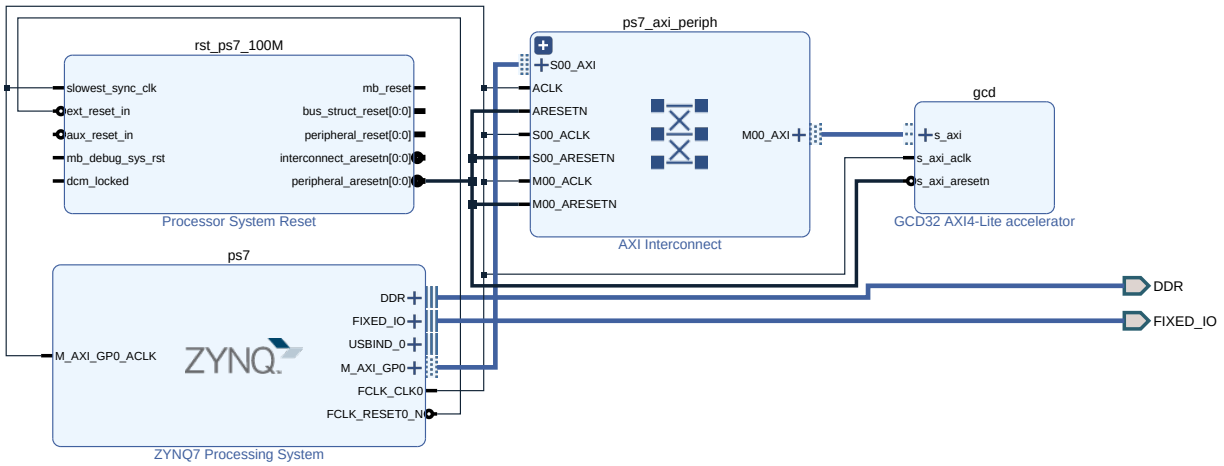


Fig. 2. Vivado block design used for both cores on the Zybo Z7-20, shown here with the Stein (`gcd32`) IP. The FSMD system is identical except for this custom IP block.

TABLE II
STEIN (`gcd32`) VERSUS FSMD (`fsmd_gcd32`)

	Stein	FSMD	Better
Processing time (cycles at 100 MHz)			
$\gcd(14, 161)$	7	18	Stein, 2.57×
Worst case, 32-bit operands	64	4.29×10^9	Stein
Mean, 10,000 board vectors	35.8	131.5	Stein, 3.68×
Time per call on board (ns)	1,180	2,173	Stein, 1.84×
Circuit size (post-route, out of context)			
LUTs	259	110	FSMD, −58%
Slices	73	38	FSMD, −48%
Flip-flops	103	98	FSMD, −5
BRAM, DSP	0, 0	0, 0	–
$F_{\max}$ (MHz)	215	215	tie
LUTs × mean cycles	9,262	14,466	Stein, 1.56×

Vivado 2021.1 for the XC7Z020 (out of context, with a 10 ns clock constraint).

Processing time: Stein (`gcd32`) wins.

- For $\gcd(14, 161)$, Stein needs only 7 cycles, while the FSMD needs 18.
- In the worst case, Stein never goes beyond 64 cycles (0.64 µs), whereas the FSMD can take more than 4 billion cycles ($2^{32} + 4 \approx 4.29 \times 10^9$, about 43 s at 100 MHz).
- On the Zybo Z7-20, one call takes 1,180 ns with Stein and 2,173 ns with the FSMD.

Circuit size: the FSMD (`fsmd_gcd32`) wins.

- The FSMD is clearly smaller and lighter on resources: it has one subtractor instead of two, a simple swap instead of shift multiplexers, and neither the counter k nor the barrel shifter that restores $a \cdot 2^k$.
- LUTs: 110 for the FSMD against 259 for Stein, 58% fewer.
- Slices: 38 against 73, 48% fewer.
- Flip-flops: 98 against 103, five fewer, which is exactly the width of Stein's counter k.

The FSMD is not always the slower one, though. It beats Stein on 14 of the 10,000 board vectors, all of them pairs with many common factors of two and small quotients. For example, the FSMD finishes $\gcd(2^{31}, 2^{30})$ in 5 cycles, while Stein needs 33.

VI. Conclusion

Both designs compute $\gcd(14, 161) = 7$, in simulation as well as on the board, and both pass every test. Putting them side by side shows a clear trade-off between time and area. When speed and a guaranteed latency matter, for example in a real-time or high-throughput system, Stein's binary algorithm (`gcd32`) is the better choice: it is 2.57 times faster on $\gcd(14, 161)$ and 3.7 times faster on average over the board vectors, and it never needs more than 64 cycles, while the FSMD can need 4.29×10^9. It even gets more work done per LUT, with an area–time product 1.56 times lower. When the area budget is what really counts, the subtractive-Euclid FSMD (`fsmd_gcd32`) is the better choice: it uses 58% fewer LUTs, 48% fewer slices and five fewer flip-flops, as long as a long and operand-dependent latency is acceptable.

References

[1] J. Stein, "Computational problems associated with Racah algebra," *J. Comput. Phys.*, vol. 1, no. 3, pp. 397–405, 1967.

[2] D. E. Knuth, *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*, 3rd ed. Reading, MA, USA: Addison-Wesley, 1997, Sec. 4.5.2.

[3] R. P. Brent and H. T. Kung, "A systolic algorithm for integer GCD computation," in *Proc. 7th IEEE Symp. Comput. Arithmetic (ARITH)*, 1985, pp. 118–125.

[4] P. P. Chu, *RTL Hardware Design Using VHDL: Coding for Efficiency, Portability, and Scalability*. Hoboken, NJ, USA: Wiley-IEEE Press, 2006.

[5] Digilent, *Zybo Z7 Board Reference Manual*.

[6] Xilinx, *Zynq-7000 SoC Technical Reference Manual*, UG585.

[7] Arm, *AMBA AXI and ACE Protocol Specification*, IHI 0022.

[8] Xilinx, *Vivado Design Suite User Guide: Designing IP Subsystems Using IP Integrator*, UG994.